

August 24, 2026 at 11:13

1. Introduction. This program counts **open** Hamiltonian paths (“open tours”) of the $m \times n$ knight graph, using a **broken-profile** (frontier) transfer method that is designed from the start to be **parallel**.

The classical program for this task, Knuth’s **DYNAHAM**, stores the frontier equivalence classes in a **trie**. A trie is compact but its updates are a serial, pointer-chasing, latency-bound computation: it uses essentially none of a modern machine’s memory bandwidth or cores. Here we replace the trie by **bucket aggregation**: we emit every successor as a record, then **sort** the records so that equal canonical keys become adjacent, then **reduce** adjacent equal keys by adding their weights. Sorting is bandwidth-bound and embarrassingly parallel, so the whole transfer parallelizes.

This first version establishes and **validates the state machine** by counting open $m \times n$ tours directly on a fixed board, comparing against known values. Later sections will add the translation-invariant canonical form needed to extract the periodic transfer (the eight per-vertex edge tables of an $m = 8$ column) and the sparse matrix-vector product that scales to large n .

2. The frontier idea in brief. Number the cells column by column: cell (r, c) , with $0 \leq r < m$ and $0 \leq c < n$, is vertex $v = cm + r$; we add vertices in the order $0, 1, 2, \dots$. A partial solution after adding vertices $0..v$ is a set of knight edges forming vertex-disjoint simple paths that covers exactly the added vertices, with every added vertex of degree ≤ 2 . Interior vertices (degree 2) are forgotten; only **active** cells—those of degree < 2 that still have an unprocessed knight neighbor—are remembered, together with how the active endpoints are paired into paths, and how many tour ends have already been “frozen” (a degree-1 vertex with no future neighbor becomes a permanent end of the single open tour; at most two are allowed).

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>

<Type definitions 3>
<Global variables 4>
<Subroutines 5>
<The main program 34>
```

3. The board and knight moves. We support any m, n (validated on small boards). **NB** holds, for each cell, the list of its knight neighbours; *maxnb* is the largest neighbour vertex number, used to detect “no future neighbour”.

⟨Type definitions 3⟩ ≡

```
typedef unsigned long long u64;
```

See also sections 6 and 13.

This code is used in section 2.

4. ⟨Global variables 4⟩ ≡

```
int allow_complete; /* strip mode keeps completed tours for boundary counting */
int m, n, V; /* rows, columns, and  $V = mn$  vertices */
int NB[64 * 64][8], ND[64 * 64]; /* neighbour lists and their lengths */
int maxnb[64 * 64]; /* largest neighbour vertex, or -1 */
static const int KR[8] = {-2, -2, -1, -1, 1, 1, 2, 2};
static const int KC[8] = {-1, 1, -2, 2, -2, 2, -1, 1};
```

See also sections 14, 16, 18, and 27.

This code is used in section 2.

5. ⟨Subroutines 5⟩ ≡

```
void build_board(void)
{
    int r, c, k;
    V = m * n;
    for (c = 0; c < n; c++)
        for (r = 0; r < m; r++) {
            int v = c * m + r;
            ND[v] = 0;
            maxnb[v] = -1;
            for (k = 0; k < 8; k++) {
                int rr = r + KR[k], cc = c + KC[k];
                if (rr ≥ 0 ∧ rr < m ∧ cc ≥ 0 ∧ cc < n) {
                    int u = cc * m + rr;
                    NB[v][ND[v]++] = u;
                    if (u > maxnb[v]) maxnb[v] = u;
                }
            }
        }
}
```

See also sections 7, 8, 9, 10, 11, 12, 15, 17, 19, 24, 26, 28, 29, 30, 31, 32, and 33.

This code is used in section 2.

6. Frontier states. A state is a small structure. nc active cells are kept sorted by vertex number in $ac[]$; $dg[i]$ is the degree (0 or 1) of $ac[i]$; $mt[i]$ is the **mate** of $ac[i]$: the vertex number of the other endpoint of its path, or the sentinel **SELF** (an isolated degree-0 cell, a length-0 path whose two ends coincide), or **FROZEN** (its path's other end is already a frozen tour end). e counts frozen tour ends (0, 1, 2); cl counts completed tours (must stay 0 except for the single final tour).

```
#define SELF (-1)
#define FROZEN (-2)
#define MAXC 40 /* maximum active cells we ever need */
⟨Type definitions 3⟩ +=
    typedef struct {
        int e, cl, nc;
        int ac[MAXC], dg[MAXC], mt[MAXC];
    } State;
```

7. Mates are stored by vertex number so that they survive cell removal. To find a cell's index we search (nc is tiny). idx returns -1 if absent.

```
⟨Subroutines 5⟩ +=
    int idx(State *s, int cell)
    {
        int i;
        for (i = 0; i < s->nc; i++)
            if (s->ac[i] == cell) return i;
        return -1;
    }
```

8. Removing an active cell (it just became interior, or left the frontier).

```
⟨Subroutines 5⟩ +=
    void del_cell(State *s, int cell)
    {
        int i = idx(s, cell);
        if (i < 0) return;
        int j;
        for (j = i + 1; j < s->nc; j++) {
            s->ac[j - 1] = s->ac[j];
            s->dg[j - 1] = s->dg[j];
            s->mt[j - 1] = s->mt[j];
        }
        s->nc--;
    }
```

9. Accessors for the mate stored at a given cell.

⟨Subroutines 5⟩ +≡

```

int get_mt(State *s, int cell)
{
    int i = idx(s, cell);
    return i < 0 ? SELF : s→mt[i];
}

void set_mt(State *s, int cell, int val)
{
    int i = idx(s, cell);
    if (i ≥ 0) s→mt[i] = val;
}

int get_dg(State *s, int cell)
{
    int i = idx(s, cell);
    return i < 0 ? 0 : s→dg[i];
}

```

10. The canonical key. The key is a byte string chosen so that lexicographic (*memcmp*) order is a canonical order and equal states get identical keys. Active cells are already sorted by vertex number; we emit, for each, its column, row, degree, and a mate code: 255 for **SELF**, 254 for **FROZEN**, else the mate's index in the sorted active list. A two-byte header carries *e* and *nc*. (For this direct counter we use **absolute** columns, which is correct on a bounded board; translation-invariant columns, needed for the periodic transfer, come later.)

```
#define KEYMAX (2 + 4 * MAXC)
```

```
<Subroutines 5> +=
```

```
int canon(State *s, unsigned char *key)
{
    int i, p = 0;
    key[p++] = (unsigned char) s->e;
    key[p++] = (unsigned char) s->nc;
    for (i = 0; i < s->nc; i++) {
        int cell = s->ac[i];
        key[p++] = (unsigned char)(cell / m);    /* column */
        key[p++] = (unsigned char)(cell % m);    /* row */
        key[p++] = (unsigned char) s->dg[i];
        if (s->mt[i] == SELF) key[p++] = 255;
        else if (s->mt[i] == FROZEN) key[p++] = 254;
        else key[p++] = (unsigned char) idx(s, s->mt[i]);
    }
    return p;    /* key length */
}
```

11. Generating the successors of a state. Adding vertex v to state s : v may be joined to 0, 1, or 2 of its already-processed neighbours that are active with degree < 2 (set A). For each admissible subset we build a child, splice the paths, then freeze any cell that has no future neighbour. This mirrors the validated reference kernel.

⟨Subroutines 5⟩ +=

```

int add_edges(State *in, int v, int *combo, int ksz, State *out)
{
    int t;
    *out = *in;    /* copy */    /* append v as an isolated active cell (degree 0, mate SELF) */
    {
        int i = out-nc++;
        out-ac[i] = v;
        out-dg[i] = 0;
        out-mt[i] = SELF;
    }
    for (t = 0; t < ksz; t++) {
        int u = combo[t];
        int mv = get_mt(out, v), mu = get_mt(out, u);
        /* a cycle closes if v and u are already the two ends of one path */
        if (mv ≡ u ∨ mu ≡ v) return 0;
        /* far ends of the two paths, taken before bumping the degrees */
        int vend = (mv ≡ SELF) ? v : mv;
        int uend = (mu ≡ SELF) ? u : mu;
        {
            int iv = idx(out, v), iu = idx(out, u);
            out-dg[iv]++;
            out-dg[iu]++;
        }
        /* point the two surviving ends at each other (FROZEN ends never move) */
        if (vend ≠ FROZEN) set_mt(out, vend, uend);
        if (uend ≠ FROZEN) set_mt(out, uend, vend);
    }
    /* whatever reached degree 2 becomes interior and leaves */
    if (get_dg(out, v) ≥ 2) del_cell(out, v);
    for (t = 0; t < ksz; t++)
        if (get_dg(out, combo[t]) ≥ 2) del_cell(out, combo[t]);
    return 1;
}

```

12. Freezing: any active cell with no future neighbour must leave. Degree 2 is already interior; degree 1 becomes a frozen tour end (incrementing e , and marking its partner's far end **FROZEN**); degree 0 with no future is stranded and the child is void. *last* is true only at the final vertex of the board.

⟨Subroutines 5⟩ +=

```

int freeze(State *s,int v,int last)
{
    int i = 0;
    while (i < s-nc) {
        int cell = s-ac[i];
        if (maxnb[cell] ≤ v) {
            int dg = s-dg[i], mt = s-mt[i];
            if (dg ≥ 2) {
                del_cell(s, cell);
                continue;
            }
            else if (dg ≡ 1) {
                s-e++;
                if (mt ≡ FROZEN) s-cl++;
                else if (mt ≠ SELF ∧ mt ≠ cell) set_mt(s, mt, FROZEN);
                del_cell(s, cell);
                continue;
            }
            else return 0;    /* degree 0 stranded */
        }
        i++;
    }
    if (s-e > 2 ∨ s-cl > 1) return 0;    /* A completed single path (e ≡ 2, no active cells) is a valid
        spanning tour of the columns processed so far. On a fixed board we only accept it at the very
        end; on a strip sweep we count it at each column boundary (see whole_table) and then drop it,
        so here we no longer reject cl ≡ 1. */
    if (¬allow_complete ∧ s-cl ≡ 1 ∧ ¬(last ∧ s-nc ≡ 0)) return 0;
    return 1;
}

```

13. Bucket aggregation. For one step we gather all successor records (key, weight) into a pool, sort the records by key, and merge adjacent equal keys by summing weights. Keys live back-to-back in *kpool*; *rec[]* indexes them.

⟨Type definitions 3⟩ +≡

```
typedef struct {
    int off, len;
    u64 w;
} Rec;
```

14. ⟨Global variables 4⟩ +≡

```
unsigned char *kpool;
long kpool_sz, kpool_use;
Rec *rec;
long nrec, rec_cap;
```

15. ⟨Subroutines 5⟩ +≡

```
void pool_reset(void)
{
    kpool_use = 0;
    nrec = 0;
}

void emit(unsigned char *key, int len, u64 w)
{
    if (kpool_use + len > kpool_sz) {
        kpool_sz = (kpool_sz * 2) + (len + 1024);
        kpool = realloc(kpool, kpool_sz);
    }
    if (nrec ≥ rec_cap) {
        rec_cap = rec_cap * 2 + 1024;
        rec = realloc(rec, rec_cap * sizeof(Rec));
    }
    memcpy(kpool + kpool_use, key, len);
    rec[nrec].off = kpool_use;
    rec[nrec].len = len;
    rec[nrec].w = w;
    kpool_use += len;
    nrec++;
}
```

16. Compare two records by their key bytes (length breaks ties).

⟨Global variables 4⟩ +≡

```
unsigned char *cmp_base;
```

17. ⟨Subroutines 5⟩ +≡

```
int reccmp(const void *A, const void *B)
{
    const Rec *a = A, *b = B;
    int l = a→len < b→len ? a→len : b→len;
    int d = memcmp(cmp_base + a→off, cmp_base + b→off, l);
    if (d) return d;
    return a→len - b→len;
}
```


18. The direct open-tour counter. We keep the current bucket as two parallel arrays: *cur* states with *curw* weights. For each vertex we expand every state, sort, reduce, and continue. Whenever we finish a column, the number of complete single tours equals the weight of states with no active cells and $e \equiv 2$; on a fixed board that is nonzero only at the very end, giving open $m \times n$.

⟨ Global variables 4 ⟩ +≡

```
State *cur;
u64 *curw;
long ncur, cur_cap;
```

19. ⟨ Subroutines 5 ⟩ +≡

```
u64 count_open(int mm, int nn)
{
    m = mm;
    n = nn;
    build_board();
    ⟨ Initialise the bucket with the empty state 20 ⟩;
    int v;
    for (v = 0; v < V; v++) {
        int last = (v ≡ V - 1);
        pool_reset();
        ⟨ Expand every current state at vertex v 21 ⟩;
        ⟨ Sort and reduce the pool into the next bucket 23 ⟩;
    }
    ⟨ Sum the weight of completed tours 25 ⟩;
}
```

20. ⟨ Initialise the bucket with the empty state 20 ⟩ ≡

```
{
    if (¬cur) {
        cur_cap = 1024;
        cur = malloc(cur_cap * sizeof(State));
        curw = malloc(cur_cap * sizeof(u64));
    }
    cur[0].e = 0;
    cur[0].cl = 0;
    cur[0].nc = 0;
    curw[0] = 1;
    ncur = 1;
}
```

This code is used in sections 19, 30, and 33.

21. For the current vertex we find its processed, still-open neighbours, then try every subset of size 0, 1, 2.

⟨Expand every current state at vertex v 21⟩ $\equiv$

```
{
  long si;
  for (si = 0; si < ncur; si++) {
    State *s = &cur[si];
    u64 w = curw[si];
    int A[8], na = 0, j;
    for (j = 0; j < ND[v]; j++) {
      int u = NB[v][j];
      if (u < v) {
        int iu = idx(s, u);
        if (iu ≥ 0 ∧ s-dg[iu] < 2) A[na++] = u;
      }
    }
    ⟨Try subsets of A of size 0,1,2 22⟩;
  }
}
```

This code is used in sections 19, 30, and 33.

22. ⟨Try subsets of A of size 0,1,2 22⟩ $\equiv$

```
{
  int a, b;
  State ch;
  unsigned char key[KEYMAX];
  int kl;
  int combo[2]; /* size 0 */
  if (add_edges(s, v, combo, 0, &ch) ∧ freeze(&ch, v, last)) {
    kl = canon(&ch, key);
    emit(key, kl, w);
  } /* size 1 */
  for (a = 0; a < na; a++) {
    combo[0] = A[a];
    if (add_edges(s, v, combo, 1, &ch) ∧ freeze(&ch, v, last)) {
      kl = canon(&ch, key);
      emit(key, kl, w);
    }
  } /* size 2 */
  for (a = 0; a < na; a++)
    for (b = a + 1; b < na; b++) {
      combo[0] = A[a];
      combo[1] = A[b];
      if (add_edges(s, v, combo, 2, &ch) ∧ freeze(&ch, v, last)) {
        kl = canon(&ch, key);
        emit(key, kl, w);
      }
    }
}
```

This code is used in section 21.

23. Sort the records; then walk them, summing runs of equal keys and decoding one representative per run back into a **State**.

⟨Sort and reduce the pool into the next bucket 23⟩ ≡

```
{
    long i;
    cmp_base = kpool;
    qsort(rec, nrec, sizeof(Rec), reccmp);
    ncur = 0;
    for (i = 0; i < nrec; ) {
        long j = i + 1;
        u64 sw = rec[i].w;
        while (j < nrec ∧ rec[j].len ≡ rec[i].len ∧ memcmp(kpool + rec[j].off, kpool + rec[i].off, rec[i].len) ≡ 0)
        {
            sw += rec[j].w;
            j++;
        }
        if (ncur ≥ cur_cap) {
            cur_cap *= 2;
            cur = realloc(cur, cur_cap * sizeof(State));
            curw = realloc(curw, cur_cap * sizeof(u64));
        }
        decode(kpool + rec[i].off, rec[i].len, &cur[ncur]);
        curw[ncur] = sw;
        ncur++;
        i = j;
    }
}
```

This code is used in sections 19, 30, and 33.

24. Decoding is the inverse of *canon*.

⟨Subroutines 5⟩ +≡

```
void decode(unsigned char *key, int len, State *s)
{
    int p = 0, i;
    (void) len;
    s→e = key[p++];
    s→nc = key[p++];
    s→cl = 0;
    for (i = 0; i < s→nc; i++) {
        int col = key[p++], row = key[p++];
        s→ac[i] = col * m + row;
        s→dg[i] = key[p++];
        int mc = key[p++];
        if (mc ≡ 255) s→mt[i] = SELF;
        else if (mc ≡ 254) s→mt[i] = FROZEN;
        else s→mt[i] = -1000 - mc;
    } /* second pass: resolve mate indices to vertex numbers */
    for (i = 0; i < s→nc; i++)
        if (s→mt[i] ≤ -1000) s→mt[i] = s→ac[-1000 - s→mt[i]];
}
```

25. $\langle \text{Sum the weight of completed tours } 25 \rangle \equiv$
 $\{$
 long i ;
 u64 $tot = 0$;
 for ($i = 0$; $i < ncur$; $i++$)
 if ($cur[i].nc \equiv 0 \wedge cur[i].e \equiv 2$) $tot += curw[i]$;
 return tot ;
 $\}$

This code is used in section 19.

26. Translation-invariant keys and stabilization. To reuse a transfer across columns we need a key that is invariant under sliding the frontier one column to the right: *relcanon* is *canon* with the least active column subtracted, so a bulk pattern and its shift get the same bytes. A completed tour (no active cells) collapses to a single “done” key. When the set of *relcanon* boundary keys at column c equals the set at $c - p$ (period $p \in \{1, 2\}$), the strip has reached its bulk and the transfer is periodic.

⟨ Subroutines 5 ⟩ +≡

```

int relcanon(State *s, unsigned char *key)
{
    int i, p = 0, mincol = 1 << 30;
    for (i = 0; i < s-nc; i++) {
        int cc = s-ac[i]/m;
        if (cc < mincol) mincol = cc;
    }
    if (s-nc ≡ 0) mincol = 0;
    key[p++] = (unsigned char) s-e;
    key[p++] = (unsigned char) s-nc;
    for (i = 0; i < s-nc; i++) {
        int cell = s-ac[i];
        key[p++] = (unsigned char)(cell/m - mincol); /* column relative to frontier */
        key[p++] = (unsigned char)(cell % m);
        key[p++] = (unsigned char) s-dg[i];
        if (s-mt[i] ≡ SELF) key[p++] = 255;
        else if (s-mt[i] ≡ FROZEN) key[p++] = 254;
        else key[p++] = (unsigned char) idx(s, s-mt[i]);
    }
    return p;
}

```

27. A tiny sorted set of *relcanon* keys, to compare boundary state-sets column to column. *setbuild* fills it from the current bucket; *seteq* compares two.

⟨ Global variables 4 ⟩ +≡

```

unsigned char *setpool[3];
Rec *setrec[3];
long setn[3];

```

28. $\langle \text{Subroutines 5} \rangle + \equiv$

```

long setbuild(int slot)
{
    unsigned char key[KEYMAX];
    long i;
    long use = 0, cnt = 0;
    static long cap[3] = {0, 0, 0};
    for (i = 0; i < ncur; i++) {
        int kl = relcanon(&cur[i], key);
        if (use + kl > cap[slot]) {
            cap[slot] = (cap[slot] * 2) + (kl + 4096);
            setpool[slot] = realloc(setpool[slot], cap[slot]);
        }
        if (cnt ≥ setn[slot]) {
            setrec[slot] = realloc(setrec[slot], (cnt + 1024) * sizeof(Rec));
            setn[slot] = cnt + 1024;
        }
        memcpy(setpool[slot] + use, key, kl);
        setrec[slot][cnt].off = use;
        setrec[slot][cnt].len = kl;
        setrec[slot][cnt].w = 0;
        use += kl;
        cnt++;
    }
    cmp_base = setpool[slot];
    qsort(setrec[slot], cnt, sizeof(Rec), reccmp); /* unique */
    long k = 0;
    for (i = 0; i < cnt; ) {
        long j = i + 1;
        while (j < cnt ∧ setrec[slot][j].len ≡ setrec[slot][i].len ∧ memcmp(setpool[slot] + setrec[slot][j].off,
            setpool[slot] + setrec[slot][i].off, setrec[slot][i].len) ≡ 0) j++;
        setrec[slot][k++] = setrec[slot][i];
        i = j;
    }
    setn[slot] = k;
    return k;
}

int seteq(int a, int b)
{
    if (setn[a] ≠ setn[b]) return 0;
    long i;
    for (i = 0; i < setn[a]; i++) {
        if (setrec[a][i].len ≠ setrec[b][i].len) return 0;
        if (memcmp(setpool[a] + setrec[a][i].off, setpool[b] + setrec[b][i].off, setrec[a][i].len)) return 0;
    }
    return 1;
}

```

29. Diagnostic probe: over the current bucket, count distinct states under several key variants, to see which distinctions inflate the state count. *variant*: 0=full *relcanon*; 1=drop the *e* byte; 2=treat **FROZEN** mates like **SELF**; 3=drop *e* and merge **FROZEN**. This tells us how much *e* and the frozen markers over-split versus a minimal frontier code.

⟨Subroutines 5⟩ +=

```

int relcanon_v(State *s, unsigned char *key, int variant)
{
    int i, p = 0, mincol = 1 << 30;
    for (i = 0; i < s-nc; i++) {
        int cc = s-ac[i]/m;
        if (cc < mincol) mincol = cc;
    }
    if (s-nc ≡ 0) mincol = 0;
    if (¬(variant & 1)) key[p++] = (unsigned char) s-e;
    key[p++] = (unsigned char) s-nc;
    for (i = 0; i < s-nc; i++) {
        int cell = s-ac[i];
        key[p++] = (unsigned char)(cell/m - mincol);
        key[p++] = (unsigned char)(cell % m);
        key[p++] = (unsigned char) s-dg[i];
        if (s-mt[i] ≡ SELF) key[p++] = 255;
        else if (s-mt[i] ≡ FROZEN) key[p++] = (variant & 2) ? 255 : 254;
        else key[p++] = (unsigned char) idx(s, s-mt[i]);
    }
    return p;
}

long distinct_under(int variant)
{
    unsigned char key[KEYMAX];
    long i, use = 0, cnt = 0;
    static long cap = 0;
    static unsigned char *pool = 0;
    static Rec *rc = 0;
    static long rcap = 0;
    for (i = 0; i < ncur; i++) {
        int kl = relcanon_v(&cur[i], key, variant);
        if (use + kl > cap) {
            cap = cap * 2 + kl + 65536;
            pool = realloc(pool, cap);
        }
        if (cnt ≥ rcap) {
            rcap = rcap * 2 + 65536;
            rc = realloc(rc, rcap * sizeof(Rec));
        }
        memcpy(pool + use, key, kl);
        rc[cnt].off = use;
        rc[cnt].len = kl;
        use += kl;
        cnt++;
    }
}

```

```

    cmp_base = pool;
    qsort(rc, cnt, sizeof(Rec), reccmp);
    long k = 0;
    for (i = 0; i < cnt; ) {
        long j = i + 1;
        while (j < cnt ∧ rc[j].len ≡ rc[i].len ∧ memcmp(pool + rc[j].off, pool + rc[i].off, rc[i].len) ≡ 0) j++;
        k++;
        i = j;
    }
    return k;
}

```


30. Diagnostic: build the strip and report, per column, the size of the *relcanon* boundary set and the first column at which it repeats.

⟨Subroutines 5⟩ +≡

```

void stab_check(int mm, int W)
{
    m = mm;
    n = W;
    build_board();
    allow_complete = 1;
    ⟨Initialise the bucket with the empty state 20⟩;
    int v, prevn[3] = {-1, -1, -1};
    int c0 = -1, period = 0;
    for (v = 0; v < V; v++) {
        int last = 0;
        pool_reset();
        ⟨Expand every current state at vertex v 21⟩;
        ⟨Sort and reduce the pool into the next bucket 23⟩;
        drop_completed();
        if (v/m ≥ 8 ∧ v/m ≤ 9) /* per-substep distinct in a (near-)stable column */
            printf("col%d: distinct=%ld (nc-bucket=%ld)\n", v/m, v % m, distinct_under(0),
                ncur);
        if (v % m ≡ m - 1) {
            int c = v/m + 1;
            int slot = c % 3;
            long sz = setbuild(slot);
            int rep = 0;
            if (c ≥ 1 ∧ seteq(slot, (c - 1) % 3)) {
                rep = 1;
                period = 1;
            }
            else if (c ≥ 2 ∧ seteq(slot, (c - 2) % 3)) {
                rep = 1;
                period = 2;
            }
            printf("col%d: relcanon_boundary_states=%ld s\n", c, sz, rep ? "≤ STABLE" : "");
            if (rep ∧ c0 < 0) {
                c0 = c;
                printf("probe: full=%ld drop-e=%ld merge-FROZEN=%ld both=%ld\n",
                    distinct_under(0), distinct_under(1), distinct_under(2), distinct_under(3));
            }
            (void) prevn;
        }
    }
    printf("stabilized at col%d, period%d\n", c0, period);
    allow_complete = 0;
}

```

31. Closing a board of width c : the whole table in one sweep. The direct counter above rebuilds the board for each width. But a single sweep of a wide strip yields open $m \times c$ for **every** c at once. The idea: after we finish column $c - 1$, ask “if the board ended here, how many complete single tours are there?”—i.e., force-freeze all still-active cells (their future neighbours would be gone) and count states that thereby become one Hamiltonian path.

A state closes to a valid single tour exactly when it has no active degree-0 cell (an isolated, hence uncoverable, vertex), and its endpoints form one path: either (a) two frozen ends already and nothing active; or (b) one frozen end and one active degree-1 cell whose mate is that frozen end; or (c) no frozen end and exactly two active degree-1 cells that are each other’s mates.

⟨Subroutines 5⟩ +≡

```

u64 close_weight(State *s, u64 w)
{
    int i, a = 0; /* number of active degree-1 endpoints */
    for (i = 0; i < s-nc; i++) {
        if (s-dg[i] ≡ 0) return 0;
        if (s-dg[i] ≡ 1) a++;
    }
    if (s-e + a ≠ 2) return 0;
    if (a ≡ 0) return (s-nc ≡ 0 ∧ s-e ≡ 2) ? w : 0;
    if (a ≡ 1) { /* the single active cell’s mate must be FROZEN */
        if (s-nc ≠ 1) return 0;
        return (s-e ≡ 1 ∧ s-mt[0] ≡ FROZEN) ? w : 0;
    } /* a==2: the two active cells must be mates of each other */
    if (s-nc ≠ 2 ∨ s-e ≠ 0) return 0;
    return (s-mt[0] ≡ s-ac[1] ∧ s-mt[1] ≡ s-ac[0]) ? w : 0;
}

```

32. Completed tours (no active cells) can never be extended; carrying them would spawn dead states that multiply. We drop them after every substep.

⟨Subroutines 5⟩ +≡

```

void drop_completed(void)
{
    long i, k = 0;
    for (i = 0; i < ncur; i++)
        if (cur[i].nc > 0) {
            cur[k] = cur[i];
            curw[k] = curw[i];
            k++;
        }
    ncur = k;
}

```

33. $\langle$ Subroutines 5 $\rangle + \equiv$

```

void whole_table(int mm, int W, u64 *out)
{
    /* out[c] = open m x c, for 1≤c≤W */
    m = mm;
    n = W;
    build_board();
    allow_complete = 1;
     $\langle$  Initialise the bucket with the empty state 20  $\rangle$ ;
    int v;
    for (v = 0; v < V; v++) {
        int last = 0;
        pool_reset();
         $\langle$  Expand every current state at vertex v 21  $\rangle$ ;
         $\langle$  Sort and reduce the pool into the next bucket 23  $\rangle$ ;
        if (v % m  $\equiv$  m - 1) { /* just finished a column: count spanning tours here */
            int c = v/m + 1;
            u64 tot = 0;
            long i;
            for (i = 0; i < ncur; i++) tot += close_weight(&cur[i], curw[i]);
            out[c] = tot;
        }
        drop_completed(); /* completed tours never extend; drop every substep */
    }
    allow_complete = 0;
}

```

34. The main program. Without arguments we run a battery of self-checks against known open-tour counts; the state machine is correct iff all pass.

⟨The main program 34⟩ ≡

```

int main(int argc, char *argv[])
{
    struct {
        int m, n;
        u64 exp;
    } chk[] = {{4, 5, 82}, {4, 6, 744}, {4, 7, 6378}, {5, 4, 82}, {5, 5, 864}, {5, 6, 18784}, {5, 8, 18061054_ULL}, {0, 0, 0}};
    if (argc ≡ 3) {
        int mm = atoi(argv[1]), nn = atoi(argv[2]);
        printf("open_␣dx%dx␣d_␣=%llu␣n", mm, nn, count_open(mm, nn));
        return 0;
    }
    if (argc ≡ 4 ∧ argv[1][0] ≡ 'w') {
        int mm = atoi(argv[2]), W = atoi(argv[3]);
        u64 *out = calloc(W + 2, sizeof(u64));
        int c;
        whole_table(mm, W, out);
        for (c = 1; c ≤ W; c++) printf("open_␣dx%dx␣d_␣=%llu␣n", mm, c, out[c]);
        return 0;
    }
    if (argc ≡ 4 ∧ argv[1][0] ≡ 's') {
        stab_check(atoi(argv[2]), atoi(argv[3]));
        return 0;
    }
    int i, bad = 0;
    for (i = 0; chk[i].m; i++) {
        u64 g = count_open(chk[i].m, chk[i].n);
        printf("open_␣dx%dx␣d_␣=%llu␣exp_␣%llu␣s␣n", chk[i].m, chk[i].n, g, chk[i].exp,
            g ≡ chk[i].exp ? "OK" : "FAIL");
        if (g ≠ chk[i].exp) bad++;
    }
    printf("%s␣n", bad ? "SOME_␣FAILED" : "ALL_␣OK");
    return bad ? 1 : 0;
}

```

This code is used in section 2.

A: [17](#), [21](#).

a: [17](#), [22](#), [28](#), [31](#).

ac: [6](#), [7](#), [8](#), [10](#), [11](#), [12](#), [24](#), [26](#), [29](#), [31](#).

add_edges: [11](#), [22](#).

allow_complete: [4](#), [12](#), [30](#), [33](#).

argc: [34](#).

argv: [34](#).

atoi: [34](#).

B: [17](#).

b: [17](#), [22](#), [28](#).

bad: [34](#).

build_board: [5](#), [19](#), [30](#), [33](#).

c: [5](#), [30](#), [33](#), [34](#).

calloc: [34](#).

canon: [10](#), [22](#), [24](#), [26](#).

cap: [28](#), [29](#).

cc: [5](#), [26](#), [29](#).

cell: [7](#), [8](#), [9](#), [10](#), [12](#), [26](#), [29](#).

ch: [22](#).

chk: [34](#).

cl: [6](#), [12](#), [20](#), [24](#).

close_weight: [31](#), [33](#).

cmp_base: [16](#), [17](#), [23](#), [28](#), [29](#).

cnt: [28](#), [29](#).

col: [24](#).
combo: [11](#), [22](#).
count_open: [19](#), [34](#).
cur: [18](#), [20](#), [21](#), [23](#), [25](#), [28](#), [29](#), [32](#), [33](#).
cur_cap: [18](#), [20](#), [23](#).
curw: [18](#), [20](#), [21](#), [23](#), [25](#), [32](#), [33](#).
c0: [30](#).
d: [17](#).
decode: [23](#), [24](#).
del_cell: [8](#), [11](#), [12](#).
dg: [6](#), [8](#), [9](#), [10](#), [11](#), [12](#), [21](#), [24](#), [26](#), [29](#), [31](#).
distinct_under: [29](#), [30](#).
drop_completed: [30](#), [32](#), [33](#).
e: [6](#).
emit: [15](#), [22](#).
exp: [34](#).
freeze: [12](#), [22](#).
FROZEN: [6](#), [10](#), [11](#), [12](#), [24](#), [26](#), [29](#), [31](#).
g: [34](#).
get_dg: [9](#), [11](#).
get_mt: [9](#), [11](#).
i: [7](#), [8](#), [9](#), [10](#), [11](#), [12](#), [23](#), [24](#), [25](#), [26](#), [28](#), [29](#),
[31](#), [32](#), [33](#), [34](#).
idx: [7](#), [8](#), [9](#), [10](#), [11](#), [21](#), [26](#), [29](#).
in: [11](#).
iu: [11](#), [21](#).
iv: [11](#).
j: [8](#), [21](#), [23](#), [28](#), [29](#).
k: [5](#), [28](#), [29](#), [32](#).
KC: [4](#), [5](#).
key: [10](#), [15](#), [22](#), [24](#), [26](#), [28](#), [29](#).
KEYMAX: [10](#), [22](#), [28](#), [29](#).
kl: [22](#), [28](#), [29](#).
kpool: [13](#), [14](#), [15](#), [23](#).
kpool_sz: [14](#), [15](#).
kpool_use: [14](#), [15](#).
KR: [4](#), [5](#).
ksz: [11](#).
l: [17](#).
last: [12](#), [19](#), [22](#), [30](#), [33](#).
len: [13](#), [15](#), [17](#), [23](#), [24](#), [28](#), [29](#).
m: [4](#), [34](#).
main: [34](#).
malloc: [20](#).
MAXC: [6](#), [10](#).
maxnb: [3](#), [4](#), [5](#), [12](#).
mc: [24](#).
memcmp: [10](#), [17](#), [23](#), [28](#), [29](#).
memcpy: [15](#), [28](#), [29](#).
mincol: [26](#), [29](#).
mm: [19](#), [30](#), [33](#), [34](#).
mt: [6](#), [8](#), [9](#), [10](#), [11](#), [12](#), [24](#), [26](#), [29](#), [31](#).

mu: [11](#).
mv: [11](#).
n: [4](#), [34](#).
na: [21](#), [22](#).
NB: [3](#), [4](#), [5](#), [21](#).
nc: [6](#), [7](#), [8](#), [10](#), [11](#), [12](#), [20](#), [24](#), [25](#), [26](#), [29](#), [31](#), [32](#).
ncur: [18](#), [20](#), [21](#), [23](#), [25](#), [28](#), [29](#), [30](#), [32](#), [33](#).
ND: [4](#), [5](#), [21](#).
nn: [19](#), [34](#).
nrec: [14](#), [15](#), [23](#).
off: [13](#), [15](#), [17](#), [23](#), [28](#), [29](#).
out: [11](#), [33](#), [34](#).
p: [10](#), [24](#), [26](#), [29](#).
period: [30](#).
pool: [29](#).
pool_reset: [15](#), [19](#), [30](#), [33](#).
prevn: [30](#).
printf: [30](#), [34](#).
qsort: [23](#), [28](#), [29](#).
r: [5](#).
rc: [29](#).
rcap: [29](#).
realloc: [15](#), [23](#), [28](#), [29](#).
Rec: [13](#), [14](#), [15](#), [17](#), [23](#), [27](#), [28](#), [29](#).
rec: [13](#), [14](#), [15](#), [23](#).
rec_cap: [14](#), [15](#).
reccmp: [17](#), [23](#), [28](#), [29](#).
relcanon: [26](#), [27](#), [28](#), [29](#), [30](#).
relcanon_v: [29](#).
rep: [30](#).
row: [24](#).
rr: [5](#).
s: [7](#), [8](#), [9](#), [10](#), [12](#), [21](#), [24](#), [26](#), [29](#), [31](#).
SELF: [6](#), [9](#), [10](#), [11](#), [12](#), [24](#), [26](#), [29](#).
set_mt: [9](#), [11](#), [12](#).
setbuild: [27](#), [28](#), [30](#).
seteq: [27](#), [28](#), [30](#).
setn: [27](#), [28](#).
setpool: [27](#), [28](#).
setrec: [27](#), [28](#).
si: [21](#).
slot: [28](#), [30](#).
stab_check: [30](#), [34](#).
State: [6](#), [7](#), [8](#), [9](#), [10](#), [11](#), [12](#), [18](#), [20](#), [21](#), [22](#),
[23](#), [24](#), [26](#), [29](#), [31](#).
sw: [23](#).
sz: [30](#).
t: [11](#).
tot: [25](#), [33](#).
u: [5](#), [11](#), [21](#).
uend: [11](#).
use: [28](#), [29](#).

u64: [3](#), [13](#), [15](#), [18](#), [19](#), [20](#), [21](#), [23](#), [25](#), [31](#), [33](#), [34](#).

V: [4](#).

v: [5](#), [11](#), [12](#), [19](#), [30](#), [33](#).

val: [9](#).

variant: [29](#).

vend: [11](#).

W: [30](#), [33](#), [34](#).

w: [13](#), [15](#), [21](#), [31](#).

whole_table: [12](#), [33](#), [34](#).

- ⟨ Expand every current state at vertex v 21 ⟩ Used in sections 19, 30, and 33.
- ⟨ Global variables 4, 14, 16, 18, 27 ⟩ Used in section 2.
- ⟨ Initialise the bucket with the empty state 20 ⟩ Used in sections 19, 30, and 33.
- ⟨ Sort and reduce the pool into the next bucket 23 ⟩ Used in sections 19, 30, and 33.
- ⟨ Subroutines 5, 7, 8, 9, 10, 11, 12, 15, 17, 19, 24, 26, 28, 29, 30, 31, 32, 33 ⟩ Used in section 2.
- ⟨ Sum the weight of completed tours 25 ⟩ Used in section 19.
- ⟨ The main program 34 ⟩ Used in section 2.
- ⟨ Try subsets of A of size 0,1,2 22 ⟩ Used in section 21.
- ⟨ Type definitions 3, 6, 13 ⟩ Used in section 2.

BUCKETHAM

	Section	Page
Introduction	1	1
The board and knight moves	3	2
Frontier states	6	3
The canonical key	10	5
Generating the successors of a state	11	6
Bucket aggregation	13	8
The direct open-tour counter	18	9
Translation-invariant keys and stabilization	26	13
Closing a board of width c : the whole table in one sweep	31	18
The main program	34	20